Name : Vaibhav Gupta
Div : D15C
Batch : C
Roll No : 61

MLDL Experiment 06

Aim: Apply K-Means and Hierarchical Clustering on sample datasets.

Theory:
Clustering is a foundational task in data mining and is used when you have unlabeled data and need to find hidden structures. Each algorithm has a different philosophy for what constitutes a "cluster."

1. Dataset Source

- Dataset Name: Pokemon Stats (pokemonDB)
- Source: Kaggle - Pokemon with Stats (or similar derivatives like pokemonDB_dataset.csv) ● Context: Contains statistics for different generations of Pokemon.

2. Dataset Description
The dataset contains standardized statistics for various Pokemon. For this experiment, we focus on combat statistics to group them by "power profiles." ● Target Variable: None (Unsupervised Learning).

- Selected Features (3 Numerical Predictors):
    1. HP Base: Hit Points (Health).
    2. Attack Base: Physical Attack strength.
    3. Defense Base: Physical Defense strength.

3.Mathematical Formulae - K-Means Clustering

K-Means is a centroid-based clustering algorithm. It is an iterative process that partitions the dataset into K predefined, non-overlapping clusters.

$$J = \sum_{j=1}^{K} \sum_{i=1}^{n} ||x_i^{(j)} - c_j||^2$$

- How it works:
  - Initialization: Randomly select K data points as the initial cluster centers (centroids).
  - Assignment Step: Assign each data point to the nearest centroid. The "nearness" is typically calculated using Euclidean distance.
  - Update Step: Recalculate the centroids by taking the average of all data points assigned to that cluster.
  - Repeat: Repeat the assignment and update steps until the centroids no longer change significantly, or a maximum number of iterations is reached.

Hierarchical Clustering:

Hierarchical clustering is a connectivity-based clustering algorithm that builds a hierarchy of clusters. It can be either agglomerative (bottom-up) or divisive (top-down). We use the agglomerative approach here.

- How it works:
  - Initialization: Start with each data point as a single, individual cluster.
  - Merging: Repeatedly merge the two closest clusters. The "closeness" between clusters is determined by a linkage method, such as 'ward', which minimizes the variance of the clusters being merged.

○ Termination: The process continues until all data points are part of a single, large cluster.

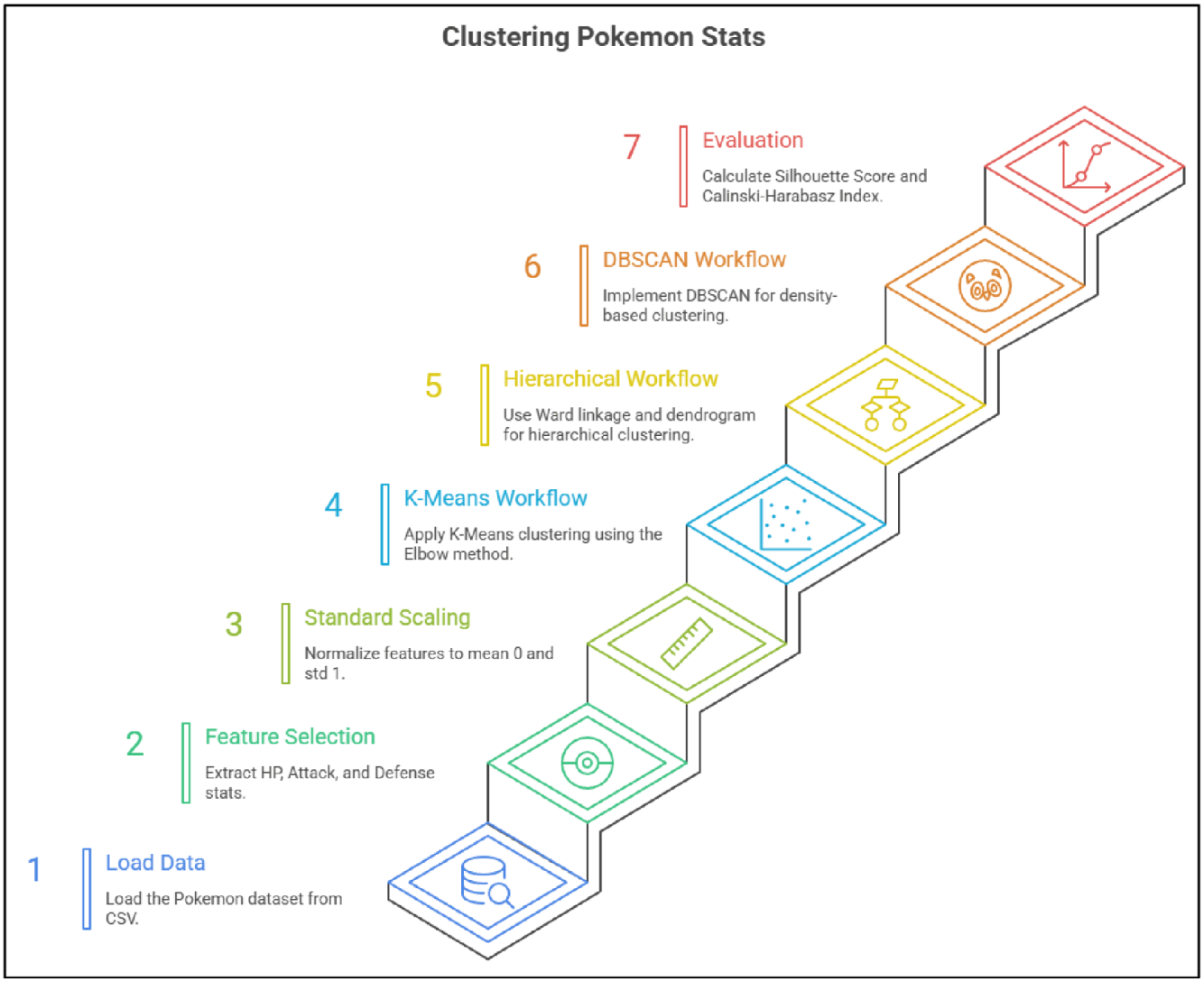
<u>DBSCAN (Density-Based Spatial Clustering of Applications with Noise):</u>

DBSCAN is a density-based algorithm that groups together points that are closely packed together, marking outliers as points that lie alone in low-density regions.

$$|N_\epsilon(p)| \geq MinPts$$

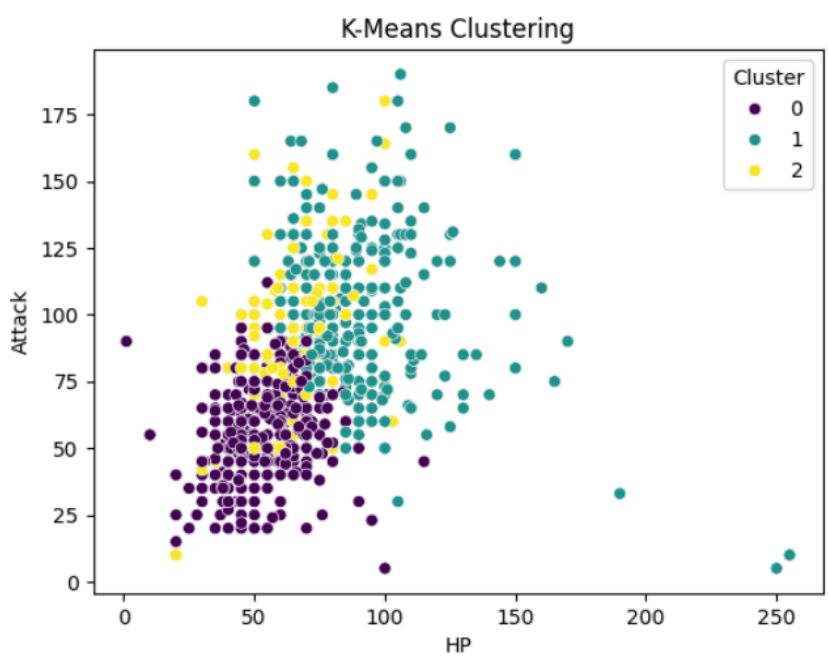
- How it works:
  - ○ Core Point: A data point is a core point if it has a minimum number of other data points (min_samples) within a specified radius (eps).
  - ○ Border Point: A border point is a data point that is within the eps distance of a core point but does not have min_samples points within its own eps radius.
  - ○ Noise Point: A noise point is a data point that is neither a core point nor a border point.

5.Methodology and workflow

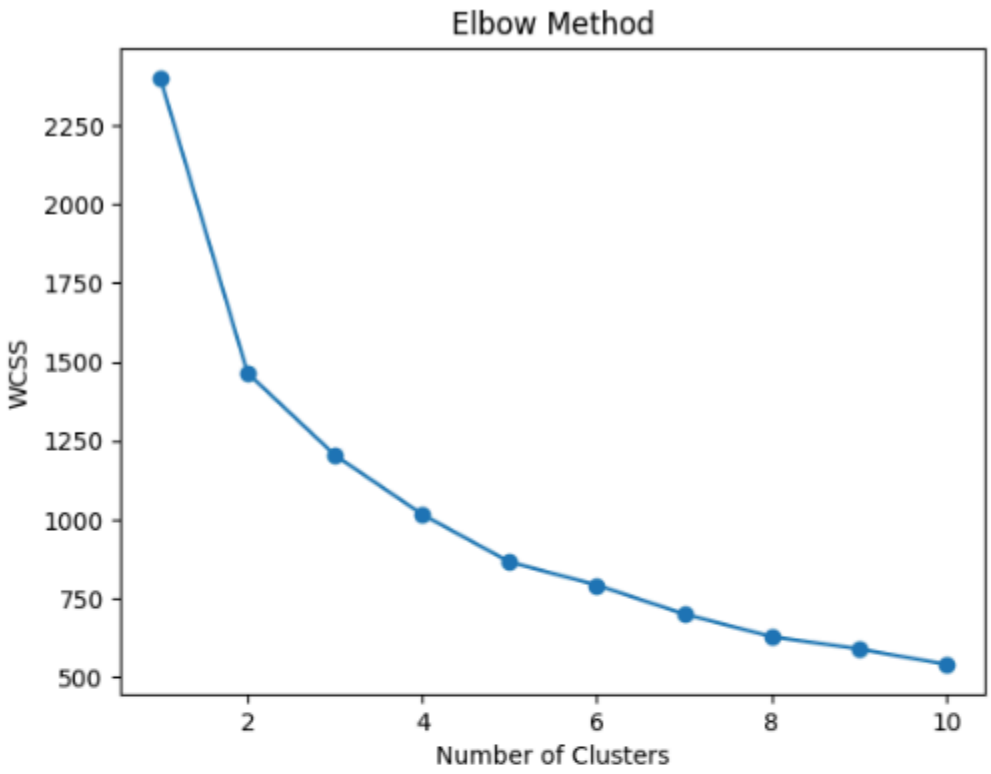


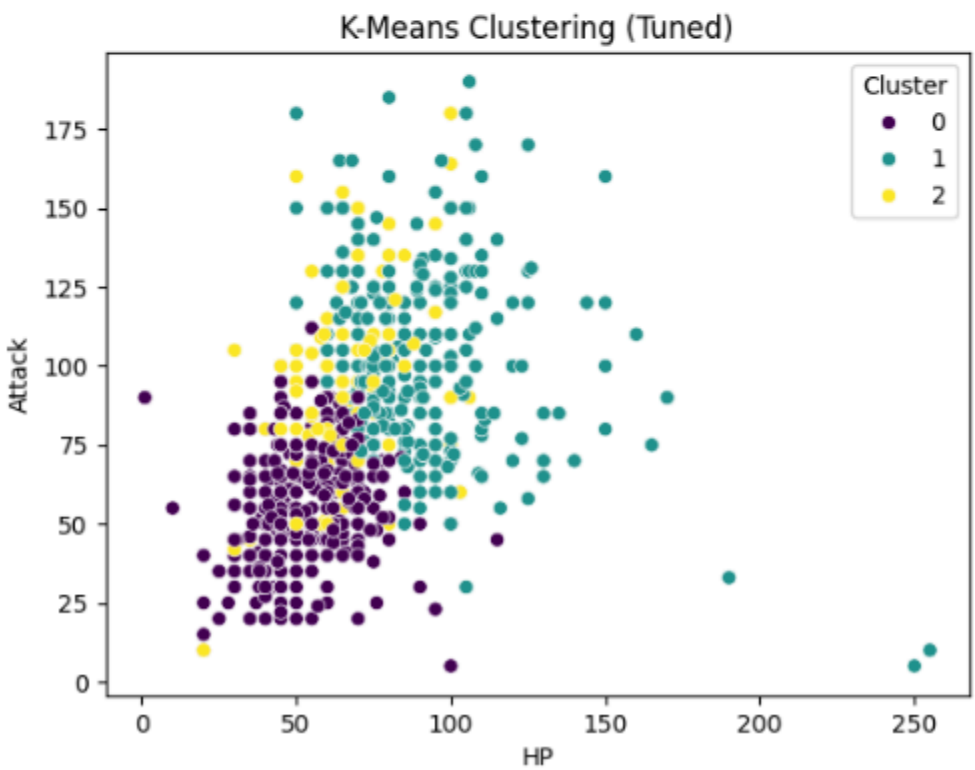
Code and Output:

```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys()))[0])

features = ['HP','Attack','Defense']
X = df[features]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = KMeans(n_clusters=3,random_state=42)

df['Cluster'] = model.fit_predict(X_scaled)

sns.scatterplot(x=X['HP'],y=X['Attack'],hue=df['Cluster'],palette='viridis')
plt.xlabel("HP")
plt.ylabel("Attack")
plt.title("K-Means Clustering")
plt.show()
```

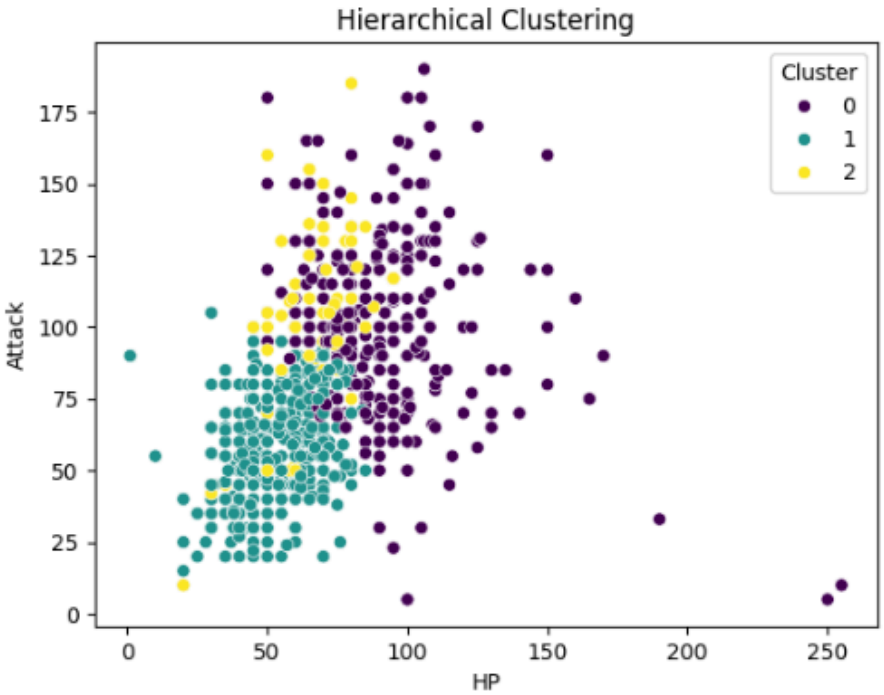


```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys()))[0])

features = ['HP','Attack','Defense']
X = df[features]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

wcss = []

for i in range(1,11):
    kmeans = KMeans(n_clusters=i,random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.plot(range(1,11),wcss,marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()

optimal_k = 3

model = KMeans(n_clusters=optimal_k,random_state=42)

df['Cluster'] = model.fit_predict(X_scaled)

sns.scatterplot(x=X['HP'],y=X['Attack'],hue=df['Cluster'],palette='viridis')
plt.xlabel("HP")
plt.ylabel("Attack")
plt.title("K-Means Clustering (Tuned)")
plt.show()
```


Elbow Method
WCSS
Number of Clusters
2250
2000
1750
1500
1250
1000
750
500
2
4
6
8
10

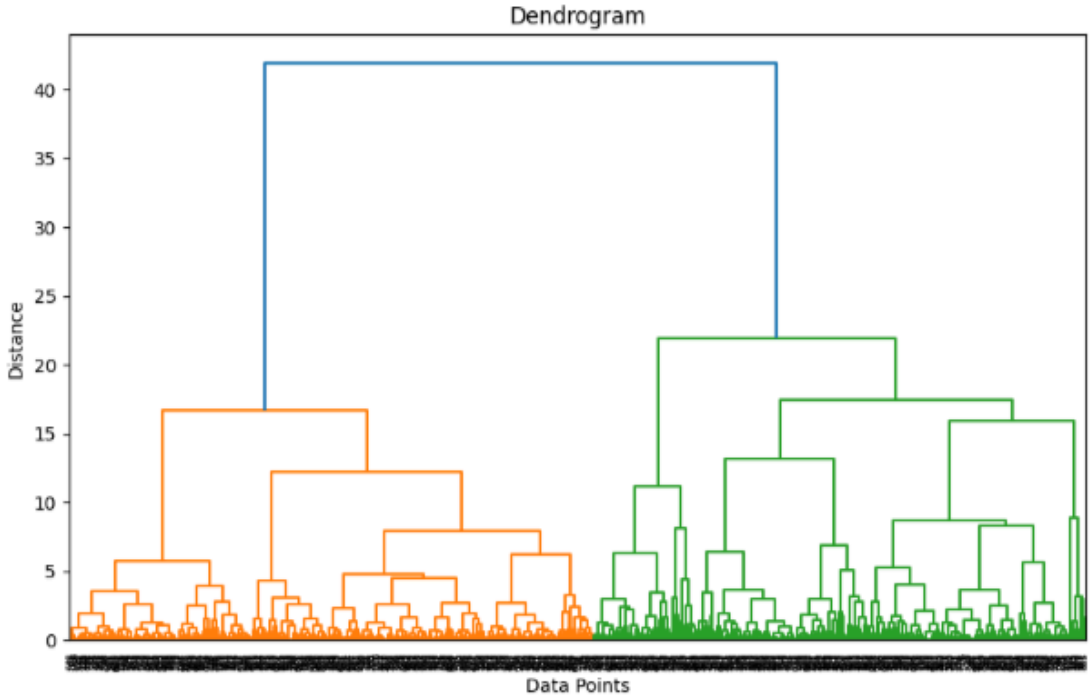

K-Means Clustering (Tuned)
Cluster
0
1
2
Attack
HP
175
150
125
100
75
50
25
0
0
50
100
150
200
250

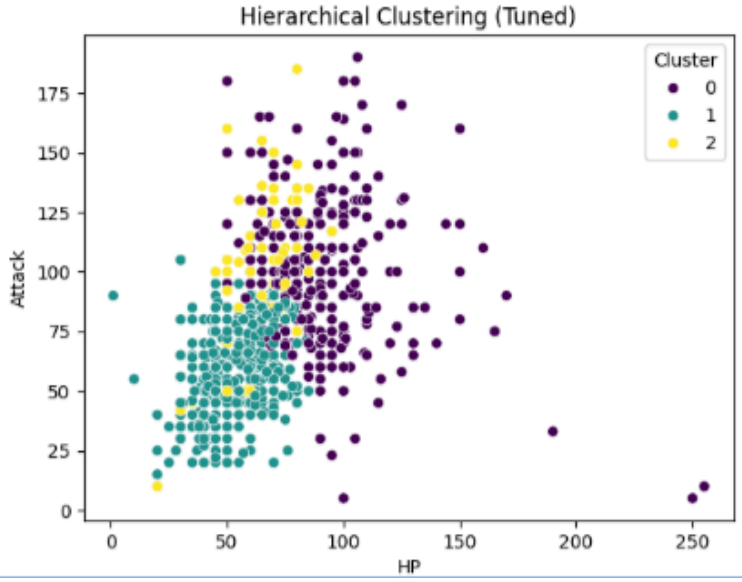
```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering

uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys()))[0])

features = ['HP','Attack','Defense']
X = df[features]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = AgglomerativeClustering(n_clusters=3)

df['Cluster'] = model.fit_predict(X_scaled)

sns.scatterplot(x=X['HP'],y=X['Attack'],hue=df['Cluster'],palette='viridis')
plt.xlabel("HP")
plt.ylabel("Attack")
plt.title("Hierarchical Clustering")
plt.show()
```




```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram,linkage

uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys()))[0])

features = ['HP','Attack','Defense']
X = df[features]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

plt.figure(figsize=(10,6))
dendrogram(linkage(X_scaled,method='ward'))
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

model = AgglomerativeClustering(n_clusters=3)

df['Cluster'] = model.fit_predict(X_scaled)

sns.scatterplot(x=X['HP'],y=X['Attack'],hue=df['Cluster'],palette='viridis')
plt.xlabel("HP")
```

```
plt.ylabel("Attack")
plt.title("Hierarchical
Clustering (Tuned)")
plt.show()
```


Dendrogram
Distance
Data Points
0
5
10
15
20
25
30
35
40


Hierarchical Clustering (Tuned)
Cluster
0
1
2
Attack
HP
0
25
50
75
100
125
150
175
0
50
100
150
200
250

Interpretation:

1. Elbow Method (K-Means Tuning):

The Elbow graph shows the relationship between the number of clusters (K) and the Within-Cluster Sum of Squares (WCSS). The WCSS decreases rapidly from K = 1 to K = 3 and then starts to level off. This indicates that adding more clusters beyond K = 3 does not significantly improve clustering performance. Hence, the optimal number of clusters is selected as K = 3.

2. K-Means Cluster Scatter Plot:

The scatter plot visualizes the clusters formed based on HP and Attack values. Each color represents a different cluster. Cluster 0 consists of Pokémon with lower HP and Attack values, Cluster 1 represents Pokémon with moderate stats, and Cluster 2 represents high HP and high Attack Pokémon (strong or legendary types). This shows that K-Means successfully groups Pokémon based on their strength levels.

3. Dendrogram (Hierarchical Clustering):

The dendrogram illustrates how clusters are formed by merging data points step by step. The height of the vertical lines represents the distance between clusters. By drawing a horizontal line across the dendrogram, we can determine the optimal number of clusters. The diagram suggests that 3 clusters provide the best separation, confirming the result obtained from the Elbow Method.

4. Hierarchical Cluster Scatter Plot:

The scatter plot shows the clusters formed using hierarchical clustering. The data points are grouped based on similarity using a bottom-up merging approach. The clusters are similar to those obtained from K-Means, although slight differences may occur due to different clustering techniques. This method provides a clear understanding of how clusters are formed.

Conclusion:

In this experiment, K-Means and Hierarchical Clustering were successfully applied to the Pokémon dataset using features such as HP, Attack, and Defense. Feature scaling was performed to ensure accurate distance-based clustering.

The Elbow Method in K-Means indicated that K = 3 is the optimal number of clusters, as the WCSS curve started flattening after this point. Similarly, the dendrogram in Hierarchical Clustering also suggested three distinct clusters, confirming the result.

The clustering results grouped Pokémon into three categories representing low, moderate, and high power levels. K-Means provided efficient and clear clustering, while Hierarchical Clustering helped visualize the cluster formation process.

Overall, the experiment demonstrated that unsupervised learning techniques can effectively identify hidden patterns in data, and both methods produced consistent and meaningful clustering results.